浙江大学

本科实验报告

课程名称: 移动平台技术开发

姓　　名: 马好君

学　　院: 计算机科学与技术学院

专　　业: 软件工程

学　　号: 3230103576

指导教师: 章国锋、陶煜波

2026 年 6 月 16 日

浙江大学实验报告

课程名称：移动平台技术开发　　实验类型：软件开发

实验项目名称：SceneZero：面向 AIGC 影视创作的 AR 场景预演与 Prompt 生成工具

学生姓名：马好君　专业：软件工程　学号：3230103576

指导老师：章国锋、陶煜波

实验地点：浙江大学玉泉校区　　实验日期：2026 年 6 月

1. 项目介绍

SceneZero 是一款面向 AIGC 电影创作前期的 iOS AR 辅助工具。它不直接生成视频，而是帮助创作者在真实空间中快速搭建电影场景参考：放置人物、道具和灯光，选择画幅、镜头、滤镜与构图辅助线，随后截取稳定的电影参考图，并自动生成结构化 Prompt 与场景元数据，保存到 Lookbook 中供后续 AI 图像或视频生成使用。

本项目的核心定位是“空间视觉参考采集器”。与 Runway、Pika 等视频生成工具相比，SceneZero 更关注生成前的构图、空间关系、灯光意图和提示词组织；与纯文字 Prompt 工具相比，它通过 AR 可视化降低了空间描述的不确定性，使创作者能够先看见场景，再把场景转化为可复用的视觉参考和 Prompt。

2. 任务概述

2.1 任务目标

完成一个可运行的 iOS AR App，支持在真实空间中放置 Subject、Prop、Light 三类影视预演元素。

- 实现 Apple Camera 风格的主界面：右侧固定快门按钮、模式选择器和当前模式调节按钮，尽量减少遮挡取景区域。
- 实现 Frame / Lens / Filter / Shot 等电影参考参数，并使最终保存图片严格来自稳定取景框区域。
- 实现 Lookbook 本地持久化：保存截图文件路径、Prompt、画幅、镜头、滤镜、灯光和物体布局元数据。
- 在课程周期内保持功能闭环，优先保证真机运行稳定、交互清晰和技术边界可解释。

开发计划围绕 AR 作业交付推进：先完成 SwiftUI + ARKit/RealityKit 的最小可运行闭环，再实现 AR 放置、手势、灯光和 Lookbook，最后集中进行 Apple 风格 UI、取景框裁切、多灯光记录、USDZ 资源和简要说明报告整理。

2.2 运行环境

开发语言与框架	Swift、SwiftUI、RealityKit、ARKit、SwiftData、CoreImage
开发工具	Xcode（课程开发环境），macOS 设备；使用真机与模拟器进行界面验证
运行平台	iOS / iPadOS，要求支持 ARKit World Tracking 与摄像头权限
测试设备	iPhone 与 iPad 真机测试；已针对横竖屏和不同屏幕尺寸调整右侧控制区
外部依赖	本次 AR 作业不依赖云端服务器；内置少量 .usdz 资源，失败时回退到基础几何体

2.3 程序功能分析与概述

- AR 场景搭建：通过平面检测或相机前方回退位置放置主体、道具和灯光，支持选中、移动、旋转、缩放等手势交互。
- 资产预设：Subject 支持 Block / Human，Prop 支持 Box / Chair / Table / Sofa 等预设，优先加载内置 USDZ，失败时生成 RealityKit 基础几何体。
- 灯光系统：支持多灯光位置和参数记录，记录方向、色温、强度、扩散范围和相对位置，并把实际放置的灯光写入 Prompt 与 Lookbook。
- 电影取景：Frame 模式统一管理画幅、镜头、镜头类型和滤镜；取景框始终是最终输出边界，构图辅助线可开关但不会写入保存图片。
- 截图管线：点击快门后异步获得 ARView 快照，按取景框裁切，再应用可选光效和 CoreImage 滤镜，最终写入沙盒文件。
- Lookbook：以画廊形式查看捕获记录，展示截图、Prompt、参数摘要，并避免在没有灯光时显示无意义的强度或扩散信息。

2.4 运行条件与限制

- 必须授予摄像头权限，设备需要支持 ARKit 世界追踪；LiDAR 不是必需条件。
- SwiftData 需要 iOS 17 或以上系统能力；AR 功能需要设备支持 ARKit World Tracking。

- 光效并非真实修改现实世界光照，而是对拍摄后图片叠加可控的视觉参考层，RealityKit 灯光主要影响虚拟物体。
- 历史场景重载保存相对布局，不采用复杂 ARWorldMap；重新打开场景时需以主体锚点作为局部坐标系参考。
- 当前版本内置模型数量有限，复杂影视资产库、在线电影参考搜索和完整期末项目化能力属于后续扩展。

3. 程序详细设计

3.1 主界面

主界面以 MainView 为入口，采用 ZStack 将 AR 相机画面、光效预览、取景框和右侧相机式控制区叠加。设计目标是让取景区域保持稳定，同时把高频操作集中在右侧，接近 Apple Camera 的操作逻辑。

层级	组件	职责
底层	ARViewContainer / ARSceneController	显示 RealityKit AR 场景，处理平面检测、物体放置、选择和手势变换。
中层	LightingOverlayView	根据已放置灯光和当前 Light 参数显示拍摄后光效预览。
中层	FramingOverlayView	显示稳定取景框、画幅遮罩和可选构图辅助线。
顶层	Top Status Capsule	显示扫描状态、Lookbook 入口、重置按钮和辅助线开关。
右侧	Camera Controls	包含当前模式调节按钮、圆形快门按钮、纵向模式选择器和向左展开的参数胶囊。

右侧模式分为 Subject、Prop、Frame、Light。Subject 与 Prop 侧重放置对象，Frame 管理画幅与视觉风格，Light 管理灯光参数。点击某一模式调节按钮时，不出现额外大面板，而是在按钮左侧短暂展开胶囊栏，显示当前参数的所有可选项或滑杆，降低界面遮挡。

3.2 Lookbook 界面

LookbookView 是捕获结果的查看和复用界面。它读取 SwiftData 中的

LookbookEntry，根据 imagePath 从沙盒读取图片，并展示 Prompt、画幅、镜头、滤镜和场景布局摘要。灯光元数据采用条件展示：如果没有放置灯光，则显示 Light: None，不展示强度和扩散；如果存在多个灯光，则展示数量和关键参数摘要。

区域	内容
画廊列表	按时间展示捕获图片和标题，作为灵感夹的基础形态。
详情页	展示大图、Prompt、Frame / Lens / Filter、Subject / Prop / Light 参数。
复用能力	Prompt 可复制；layoutJSON 可用于后续实现相对布局重载。

3.3 核心算法

本项目没有复杂数值算法，核心在于 AR 坐标管理、截图后处理和 Prompt 结构化生成。以下为关键流程伪代码：

```
onCaptureButtonTapped:
   lock capture state and show shutter feedback
   ARView.snapshot { image in
      frameRect = calculateStableFrameRect(screenSize, aspectRatio,
safeInsets)
      cropped = crop(image, frameRect)
      lighting = applyPostCaptureLighting(cropped, placedLightRecords)
      filtered = applyCoreImageFilter(lighting, selectedFilter)
      path = ImageStorageService.save(filtered)
      prompt = PromptService.build(sceneRecords, frame, lens, filter,
lights)
      SwiftData.insert(LookbookEntry(path, prompt, metadata, layoutJSON))
      unlock capture state
   }
```

历史布局保存采用相对坐标：以主体锚点作为局部坐标原点，记录其他对象相对于主体的位置、缩放、朝向和灯光参数。这样避免 ARKit 每次启动世界原点不同导致绝

对坐标漂移。

3.4 数据库设计

项目使用 SwiftData 保存结构化元数据，但不将高分辨率图片二进制直接写入数据库。截图文件由 ImageStorageService 写入 App 沙盒，SwiftData 仅保存 imagePath，避免数据库肥大化。

字段	说明
id / createdAt / title	记录唯一标识、创建时间和标题。
imagePath	截图在沙盒中的相对路径或文件名。
prompt	由 PromptService 生成的结构化 AIGC 提示词。
shotType / frameAspectRatio / lensPreset / filterName	电影取景与视觉风格参数。
lightPreset / colorTemperature / lightIntensity / lightRadius	实际灯光摘要；无灯光时不展示无意义数值。
layoutJSON	序列化后的 SceneObjectRecord 数组，用于记录物体和灯光的相对布局。

3.5 其他

- UI 风格参考 Apple Camera：使用圆形快门、纵向模式选择、半透明玻璃质感和短生命周期参数胶囊，避免传统大控制台遮挡画面。
- USDZ 模型作为专业感增强项，仍保留几何体回退，保证资源缺失时 App 不崩溃。

4. 数据结构设计

4.1 SceneObjectRecord

SceneObjectRecord 用于记录每个 AR 对象的可持久化信息。它既服务于 Prompt 生成，也服务于 Lookbook 详情与未来场景重载。

变量	描述
id	对象唯一标识。
type	对象类型，例如 subject、prop、light。
assetPreset	资产预设，例如 human、chair、table、sofa。
relativeX / relativeY / relativeZ	相对于主体局部坐标系的位置。
scale / yaw	对象缩放和朝向。
lightPreset / colorTemperature / lightIntensity / lightRadius	灯光对象专用参数。

4.2 LookbookEntry

LookbookEntry 是 SwiftData 持久化模型。它保存截图路径、Prompt 和结构化参数，是从 AR 场景到创作参考资料的核心桥梁。

变量	描述
imagePath	本地图片文件路径，不直接存储 Data。
prompt	可复制到 AIGC 工具的提示词。
frameAspectRatio / lensPreset / filterName	画幅、镜头和滤镜信息。
layoutJSON	场景对象与灯光布局的 JSON 字符串。

4.3 AssetPreset 与参数枚举

AssetPreset、LightPreset、ColorTemperature、FrameAspectRatio、LensPreset、FilterPreset 等枚举负责把 UI 选项、RealityKit 实体生成和 Prompt 文本统一起来。这样用户点击的每个选项都能对应到可见效果和可保存元数据。

枚举	示例	作用
AssetPreset	Block / Human / Chair / Table / Sofa	决定加载 USDZ 或生成几何体。
LightPreset	Key Left 45 / Rim Right Back / Top Down	决定虚拟灯位置和 Prompt 语义。
FrameAspectRatio	16:9 / 2.39:1 / 1:1 / 9:16	决定取景框比例与最终裁切。
LensPreset	24mm / 35mm / 50mm / 85mm	用于镜头参考与 Prompt 描述。
FilterPreset	Realistic / Teal Orange / Neon Night / Noir Gold	对应 CoreImage 拍摄后滤镜。

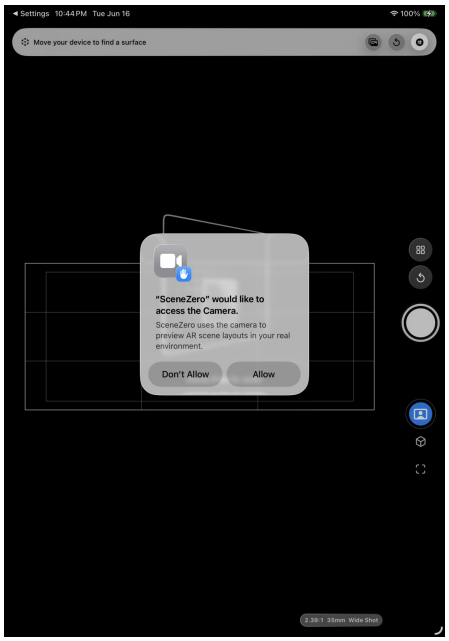
5. 运行结果

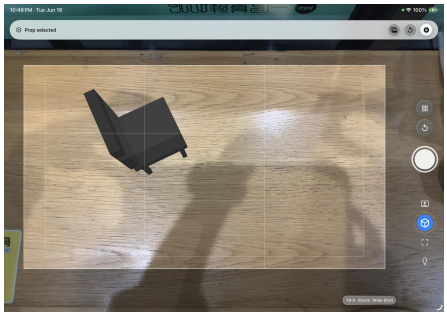
运行后 App 首先请求摄像头权限并进入 AR 取景界面。顶部状态胶囊显示平面检测状态，例如 No surface found yet 或 anchor placed。右侧控制区始终保持在屏幕右侧：中部为圆形快门按钮，下方为 Subject / Prop / Frame / Light 模式选择器，上方为当前模式调节按钮。

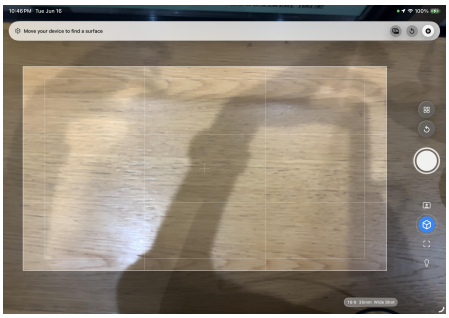
在横屏和竖屏下，控制区均保持右侧纵向布局，避免早期版本中底部面板遮挡大面积画面的问题。取景框线避让右侧常驻控制区，并作为最终截图裁切边界。

5.1 主界面

可显示真实相机画面和 AR 虚拟对象；可切换模式并调节参数。

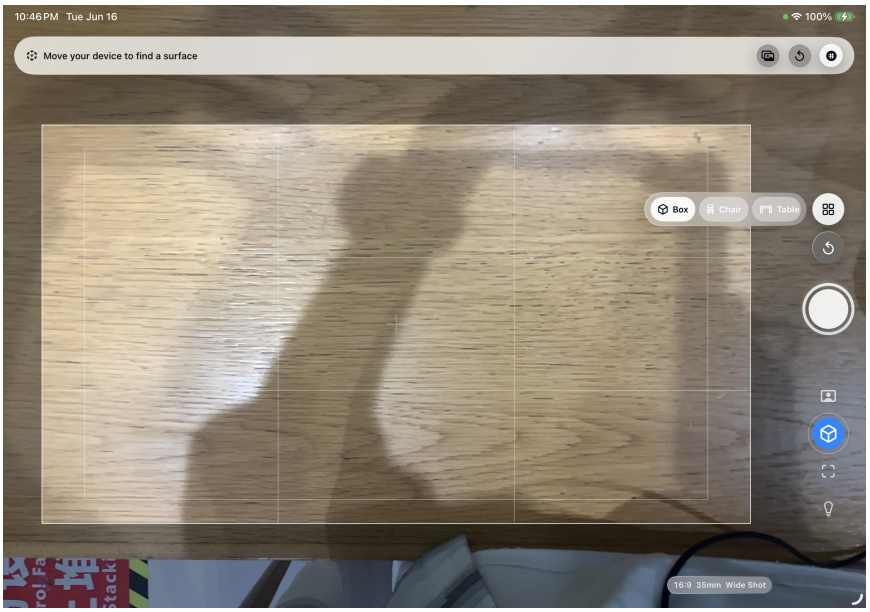






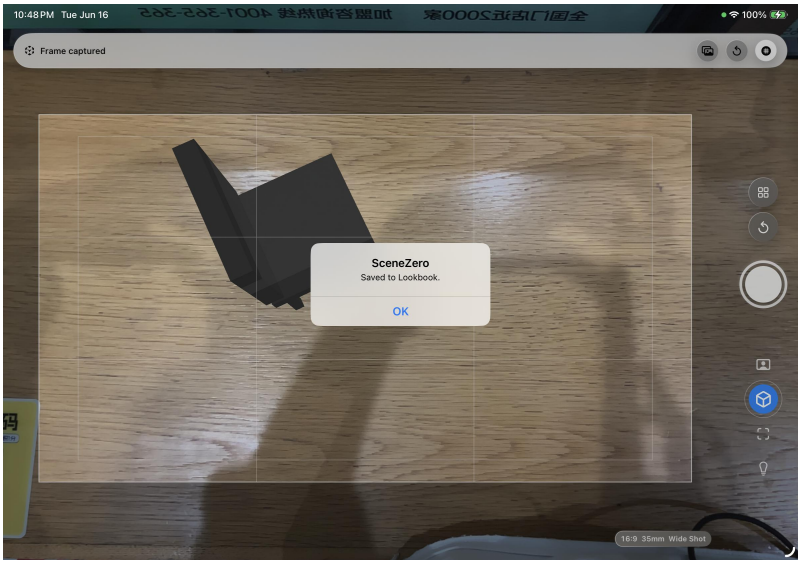
5.2 参数胶囊

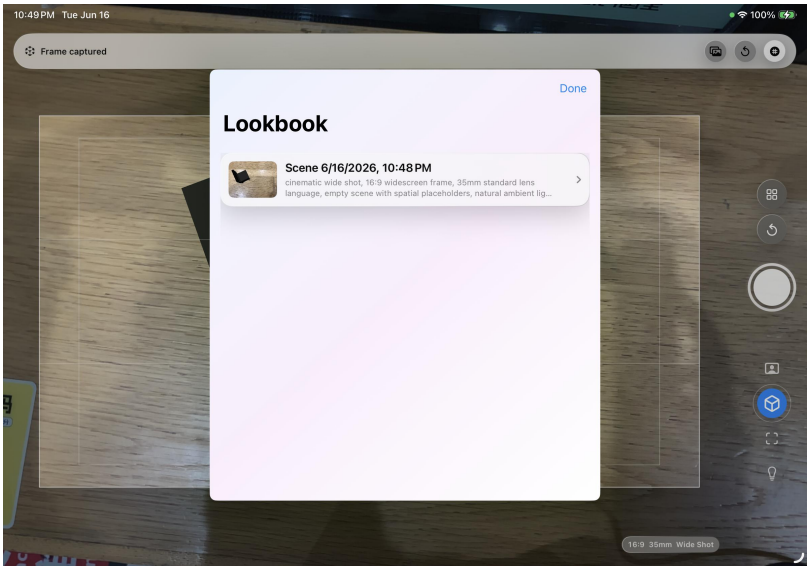
点击当前模式调节按钮后，选项在按钮左侧展开，短暂覆盖取景区域但不改变最终裁切边界。

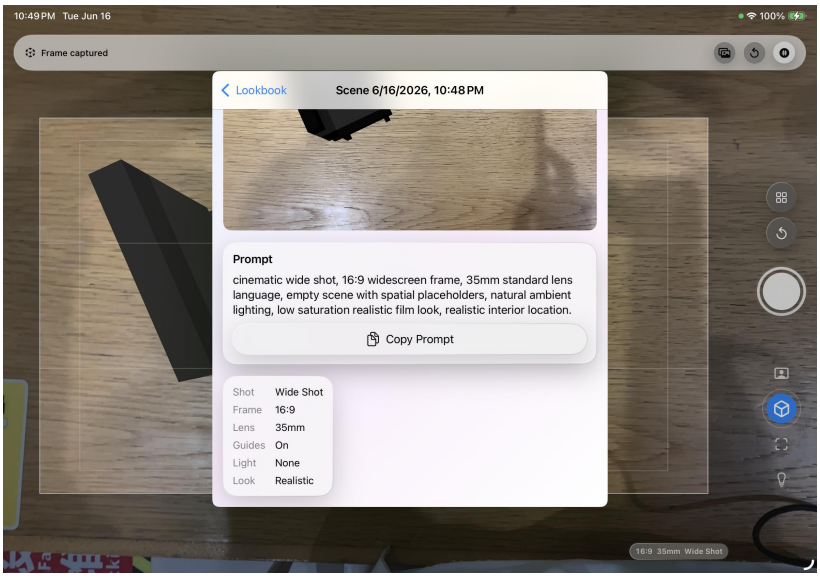


5.3 Lookbook

可查看已保存截图、Prompt 与元数据，支持继续扩展为灵感夹分区。


10:48 PM Tue Jun 16
100%
Frame captured
SceneZero
Saved to Lookbook.
OK
16:9 35mm Wide Shot


10:49 PM Tue Jun 16
100%
Frame captured
Done
Lookbook
Scene 6/16/2026, 10:48 PM
cinematic wide shot, 16:9 widescreen frame, 35mm standard lens language, empty scene with spatial placeholders, natural ambient lig...
16:9 35mm Wide Shot


10:49 PM Tue Jun 16
100%
Frame captured
Lookbook
Scene 6/16/2026, 10:48 PM
Prompt
cinematic wide shot, 16:9 widescreen frame, 35mm standard lens language, empty scene with spatial placeholders, natural ambient lighting, low saturation realistic film look, realistic interior location.
Copy Prompt
Shot Wide Shot
Frame 16:9
Lens 35mm
Guides On
Light None
Look Realistic
16:9 35mm Wide Shot

6. 讨论与总结

本次 AR 编程作业从最初的 AR 场景辅助设想逐步收敛为一个可运行、可解释、可迭代的 iOS App。最大的工程收获是认识到 AR 与 AIGC 结合时，重点不一定是直接生成视频，而是把真实空间中的构图、布光和道具关系转化为稳定的视觉参考与结构化 Prompt。

开发过程中遇到的主要挑战包括：RealityKit 与 CoreImage 的实时融合不适合 MVP，因此改为拍摄后滤镜；RealityKit 灯光不能直接改变真实画面，因此采用虚拟灯光记录加拍摄后光效预览；SwiftData 不适合直接保存大图，因此改为文件路径加元数据；AR 世界坐标在每次启动时不稳定，因此历史布局采用相对坐标思路。

UI 方面，早期大面板容易遮挡取景画面，后续参考 Apple Camera 改为右侧控制列、圆形快门、纵向模式选择和轻量参数胶囊，交互明显更适合 AR 拍摄场景。项目仍有改进空间，例如加入更多高质量 USDZ 资产、完善多界面灵感夹、支持电影场景参考搜索，以及在新版本 iOS SDK 中进一步使用原生玻璃材质 API。

总体而言，SceneZero 已完成 AR 编程作业所需的核心闭环：在真实空间中搭建场景、调整电影参数、捕获稳定参考图、保存 Lookbook 并生成可复用 Prompt。该闭环也为后续扩展为更完整的 AIGC 影视创作辅助工具打下了基础。